AppCompraVenta

Documentacion de Cambios

Commits: 7dcd5aa y 983f114 | Abril 2026

Introduccion

Este documento describe paso a paso los cambios implementados en la aplicacion AppCompraVenta despues del commit 'crearanuncio con 3 fotos y descripcion' (bf16253). Los cambios abarcan dos commits:

- 7dcd5aa — Like/Dislike en anuncios + sistema de comentarios (1 por usuario)
- 983f114 — Likes en comentarios y soporte para respuestas anidadas

La guia esta escrita en formato 'hazlo conmigo': cada seccion indica exactamente en que archivo trabajar y que agregar o modificar.

Commit 1 de 2

7dcd5aa	like and dislike on main ad, comments 1 per user	13 Abr 2026

En este commit se agregaron dos grandes funcionalidades a la pantalla de detalle de un anuncio: el sistema de reacciones (Like/Dislike) y el sistema de comentarios con un limite de 1 comentario por usuario.

Archivos afectados

Archivo	Tipo	Descripcion
`ModeloComentario.kt`	NUEVO	Clase de datos para un comentario
`AdaptadorComentario.kt`	NUEVO	RecyclerView adapter para la lista de comentarios
`DetalleAnuncio.kt`	MODIFICADO	Activity principal del anuncio
`activity_detalle_anuncio.xml`	MODIFICADO	Layout del detalle del anuncio
`item_comentario.xml`	NUEVO	Layout de cada fila de comentario
`res/drawable/`	NUEVO (x4)	Iconos de like/dislike llenos y contorneados

Paso 1 — Crear los iconos en res/drawable

Antes de tocar codigo, necesitas tener los 4 drawables que se usan para los botones de reaccion. Agrega estos archivos PNG (o vectores) en la carpeta:

```
app/src/main/res/drawable/
```

- ic_like_outline.png — pulgar arriba sin rellenar
- ic_like_filled.png — pulgar arriba relleno (activo)
- ic_dislike_outline.png — pulgar abajo sin rellenar
- ic_dislike_filled.png — pulgar abajo relleno (activo)
- ic_person_outline.png — icono generico de usuario (para comentarios)
- shape_circle_gray.png — fondo circular gris (para avatar)

> *Tip: Puedes descargarlos gratis de Material Icons (fonts.google.com/icons). Busca 'thumb_up' y 'thumb_down'.*

Paso 2 — Crear ModeloComentario.kt

Crea el archivo de modelo de datos en:

```
app/src/main/java/com/example/appcomprayventa/Modelo/ModeloComentario.kt
```

Este modelo le dice a Firebase como mapear cada comentario de la base de datos a un objeto Kotlin. Debe tener:

- uid: String — ID del usuario que escribio el comentario
- comentario: String — El texto del comentario
- tiempo: Long — Timestamp en milisegundos
- Un constructor vacio (obligatorio para Firebase Realtime Database)

> *IMPORTANTE: El constructor vacio es requerido por Firebase. Sin el, la app crasheara al intentar leer comentarios de la base de datos.*

Paso 3 — Actualizar el layout activity_detalle_anuncio.xml

Abre el archivo:

`app/src/main/res/layout/activity_detalle_anuncio.xml`

Haz los siguientes cambios en el XML:

3a. Cambiar ScrollView por NestedScrollView

La etiqueta raiz debe cambiar de ScrollView a NestedScrollView. Esto es necesario para que el RecyclerView de comentarios pueda coexistir sin conflictos de scroll:

- Cambia la etiqueta de apertura: de <ScrollView a <androidx.core.widget.NestedScrollView
- Agrega el namespace: xmlns:app="http://schemas.android.com/apk/res-auto"
- Cambia la etiqueta de cierre al final del archivo: de </ScrollView> a </androidx.core.widget.NestedScrollView>

3b. Agregar bloque de Likes/Dislikes

Despues del TextView que muestra el precio (o la descripcion), agrega un LinearLayout horizontal centrado con dos bloques: uno para el boton de like con su contador, y otro para el dislike con su contador. Los IDs que debes usar son:

- Btn_like — ImageButton del like
- Tv_likes_count — TextView contador de likes
- Btn_dislike — ImageButton del dislike
- Tv_dislikes_count — TextView contador de dislikes

3c. Agregar seccion de Comentarios

Debajo del bloque de likes, agrega una linea divisora (View de 1dp), un titulo 'Comentarios', y luego:

- Un LinearLayout horizontal con un EditText (ID: Et_comentario) y un Button (ID: Btn_enviar_comentario)
- Un RecyclerView (ID: RV_Comentarios) con nestedScrollingEnabled='false' y el atributo app:layoutManager de LinearLayoutManager

> *El atributo nestedScrollingEnabled='false' en el RecyclerView es clave. Evita que el RecyclerView tenga su propio scroll y deja que el NestedScrollView padre se encargue de todo.*

Paso 4 — Crear el layout item_comentario.xml

Crea el archivo de layout para cada fila de comentario en:

```
app/src/main/res/layout/item_comentario.xml
```

Este layout define como se ve cada comentario individualmente. Debe contener:

- Un icono de usuario (ImageView) + nombre del usuario (Tv_usuario_comentario) + fecha (Tv_fecha_comentario)
- El texto del comentario (Tv_texto_comentario)
- Un boton de like (Btn_like_comentario) + contador (Tv_likes_comentario)
- Un boton de texto 'Responder' (Btn_responder_comentario) — es un TextView con estilo de link
- Un RecyclerView anidado (RV_Respuestas) — para mostrar las respuestas a este comentario

> *El RecyclerView de respuestas dentro del item del comentario es lo que habilita el sistema de respuestas anidadas del siguiente commit.*

Paso 5 — Crear AdaptadorComentario.kt

Crea el adaptador en:

```
app/src/main/java/com/example/appcomprayventa/Adaptadores/AdaptadorComentario.kt
```

Este adaptador recibe tres parametros en el constructor:

- context: Context
- comentariosArrayList: ArrayList<ModeloComentario>
- idAnuncio: String — necesario para construir la ruta correcta en Firebase

Funciones que debe implementar:

1. cargarInformacionUsuario() — consulta Firebase con el uid del comentario para obtener el nombre del usuario y mostrarlo en tvUsuario
2. manejarLikesComentario() — escucha en tiempo real el nodo Likes dentro del comentario. Cambia el icono segun si el usuario ya dio like o no. Al hacer clic, agrega o elimina su uid del nodo Likes
3. mostrarDialogoRespuesta() — muestra un AlertDialog con un EditText para escribir una respuesta
4. guardarRespuestaEnFirebase() — guarda la respuesta usando el uid del usuario como clave, garantizando 1 respuesta por usuario por comentario
5. cargarRespuestas() — configura y alimenta el RecyclerView anidado de respuestas (RV_Respuestas) con AdaptadorRespuesta

Paso 6 — Modificar DetalleAnuncio.kt

Abre el Activity en:

`app/src/main/java/com/example/appcomprayventa/DetalleAnuncio.kt`

6a. Agregar variables de instancia

Al inicio de la clase, junto a la variable idAnuncio existente, agrega:

- miReaccion: String = "" — almacena si el usuario actual dio 'like', 'dislike' o nada
- comentariosArrayList: ArrayList<ModeloComentario>
- adaptadorComentario: AdaptadorComentario

6b. Llamar las nuevas funciones desde onCreate()

Despues de la llamada existente a cargarImagenesCarrusel(), agrega dos nuevas llamadas:

- manejarReacciones() — inicializa el sistema de likes/dislikes
- Configura el listener del boton Btn_enviar_comentario para llamar validarYGuardarComentario()
- cargarComentarios() — inicializa el RecyclerView y escucha los comentarios en Firebase

6c. Implementar manejarReacciones()

Esta funcion escucha en tiempo real el nodo Reacciones del anuncio en Firebase. La logica es:

- Recorre todos los hijos del nodo. Cuenta cuantos son 'like' y cuantos son 'dislike'
- Detecta si el uid del usuario actual ya tiene una reaccion guardada
- Actualiza los contadores en pantalla y llama a actualizarIconos()

6d. Implementar reaccionar(nuevoTipo)

Funcion con logica de toggle: si el usuario hace clic en el mismo boton que ya presiono, elimina su reaccion. Si hace clic en el contrario, sobreescribe con el nuevo valor.

6e. Implementar actualizarIconos()

Cambia los drawables de los botones segun el valor de miReaccion:

- miReaccion == 'like': like relleno + dislike contorno
- miReaccion == 'dislike': like contorno + dislike relleno
- miReaccion == '' (vacio): ambos en contorno

6f. Implementar validarYGuardarComentario()

Antes de guardar, valida dos cosas: que el texto no este vacio, y que no exceda 50 palabras (divide el texto con \s+ para contar). Si pasa la validacion, llama a guardarComentarioEnFirebase().

6g. Implementar guardarComentarioEnFirebase()

Guarda el comentario en la ruta: Anuncios/{idAnuncio}/Comentarios/{uid}. Al usar el uid del usuario como clave, Firebase sobreescribe automaticamente si el usuario ya tenia un comentario — asi se garantiza el limite de 1 por usuario.

6h. Implementar cargarComentarios()

Escucha en tiempo real el nodo Comentarios. En cada update limpia la lista, parsea los hijos como ModeloComentario, y actualiza el adaptador en el RV_Comentarios.

Estructura resultante en Firebase

Al final de este commit, la estructura de tu base de datos para un anuncio deberia verse asi:

```
Anuncios/
  {idAnuncio}/
    uid: 'abc123'
    titulo: 'Mi anuncio'
    Reacciones/
      {uidUsuario1}: 'like'
      {uidUsuario2}: 'dislike'
    Comentarios/
      {uidUsuario1}/
        uid: 'uidUsuario1'
        comentario: 'Texto del comentario'
        tiempo: 1713000000000
```

Commit 2 de 2

983f114	comments in comments with likes	13 Abr 2026

Este commit expande el sistema de comentarios: ahora los comentarios tienen likes propios, y cada comentario admite respuestas que tambien tienen likes. Ademas se corrigieron crasheos en fragmentos por listeners de Firebase que se disparaban con el fragment ya destruido.

Archivos afectados

Archivo	Tipo	Descripcion
`ModeloRespuesta.kt`	NUEVO	Clase de datos para una respuesta
`AdaptadorRespuesta.kt`	NUEVO	Adapter para RecyclerView de respuestas
`AdaptadorComentario.kt`	MODIFICADO	Se activan los likes y respuestas
`item_respuesta.xml`	NUEVO	Layout de cada fila de respuesta
`FragmentCuenta.kt`	MODIFICADO	Fix anti-crasheo por zombie listener
`FragmentMisAnuncios.kt`	MODIFICADO	Fix anti-crasheo + se carga todos los anuncios

Paso 1 — Crear ModeloRespuesta.kt

Crea el modelo en:

`app/src/main/java/com/example/appcomprayventa/Modelo/ModeloRespuesta.kt`

Identico en estructura a ModeloComentario, pero con el campo 'respuesta' en lugar de 'comentario':

- uid: String
- respuesta: String
- tiempo: Long
- Constructor vacio (obligatorio para Firebase)

Paso 2 — Crear item_respuesta.xml

Crea el layout en:

`app/src/main/res/layout/item_respuesta.xml`

Es un layout mas compacto que item_comentario (sin boton de 'Responder' ni RecyclerView anidado). Contiene:

- Icono pequeno de usuario (20dp) + nombre (Tv_usuario_respuesta) + fecha (Tv_fecha_respuesta) — todos en la misma fila

- Texto de la respuesta (Tv_texto_respuesta)
- Boton de like (Btn_like_respuesta) + contador (Tv_likes_respuesta)

> *El fondo del item debe ser #FAFAFA para distinguirse visualmente del comentario padre que tiene fondo blanco.*

Paso 3 — Crear AdaptadorRespuesta.kt

Crea el adaptador en:

```
app/src/main/java/com/example/appcomprayventa/Adaptadores/AdaptadorRespuesta.kt
```

El constructor recibe 4 parametros:

- context: Context
- respuestasArrayList: ArrayList<ModeloRespuesta>
- idAnuncio: String — para construir la ruta en Firebase
- uidComentarioPadre: String — para navegar al nodo correcto de Respuestas

Funciones que debe implementar:

6. cargarInformacionUsuario() — igual que en AdaptadorComentario, consulta el nombre del usuario por uid
7. manejarLikes() — escucha el nodo Likes dentro de la respuesta. La ruta completa es: Anuncios/{idAnuncio}/Comentarios/{uidComentarioPadre}/Respuestas/{uidRespuesta}/Likes. Logica de toggle identica al like de comentarios

Paso 4 — Corregir FragmentCuenta.kt (Fix anti-crasheo)

Abre el fragment en:

```
app/src/main/java/com/example/appcomprayventa/Fragmentos/FragmentCuenta.kt
```

El problema: el listener de Firebase en onDataChange() intentaba usar 'binding' despues de que el fragment ya habia sido destruido (por ejemplo, al navegar a otra pantalla), lo cual causaba un NullPointerException.

La solucion es agregar un 'escudo' al inicio de onDataChange() y onCancelled():

```
if (!isAdded || _binding == null) return
```

Esto verifica que el fragment todavia esta adjunto a su Activity y que el binding no ha sido limpiado antes de intentar usar cualquier vista.

> *Esta es una buena practica para CUALQUIER listener de Firebase que se registre en un Fragment. Aplicala siempre.*

Paso 5 — Corregir FragmentMisAnuncios.kt

Abre el fragment en:

```
app/src/main/java/com/example/appcomprayventa/Fragmentos/FragmentMisAnuncios.kt
```

5a. Fix anti-crasheo

Al igual que en FragmentCuenta, agrega al inicio de onDataChange() y onCancelled():

```
if (!isAdded) return
```

5b. Cambio temporal: mostrar TODOS los anuncios

En este commit se comento temporalmente el filtro que mostraba solo los anuncios del usuario actual. El query cambio de:

```
ref.orderByChild("uid").equalTo(uidActual)
```

a simplemente:

```
ref.addValueEventListener(...)
```

> *ATENCION: Este cambio es temporal para pruebas. En produccion debes volver a activar el filtro por uid para que cada usuario solo vea sus propios anuncios.*

Estructura final de Firebase con respuestas

Despues de este commit, la estructura completa con respuestas y likes en comentarios queda asi:

```
Comentarios/
  {uidUsuario1}/
    uid: '...'
    comentario: 'Texto del comentario'
    tiempo: 1713000000000
    Likes/
      {uidUsuario2}: 'like'
    Respuestas/
      {uidUsuarioQueResponde}/
        uid: '...'
        respuesta: 'Texto de la respuesta'
        tiempo: 1713000001000
        Likes/
          {uidOtroUsuario}: 'like'
```

Resumen General

Estos dos commits transformaron la pantalla de detalle del anuncio de una vista de solo lectura a una pantalla interactiva completa. A continuacion un resumen de todo lo construido:

Funcionalidad	Descripcion
Like / Dislike en anuncio	Cada usuario puede reaccionar una sola vez. Toggle: si repite clic en el mismo, se quita la reaccion.
Comentarios (1 por usuario)	El usuario escribe y envia un comentario de max 50 palabras. Firebase garantiza 1 por usuario usando el uid como clave.
Likes en comentarios	Cada comentario tiene su propio contador de likes con icono toggle.
Respuestas a comentarios	Desde cada comentario se puede abrir un dialogo para responder. Limite de 1 respuesta por usuario por comentario.
Likes en respuestas	Las respuestas tambien tienen su propio sistema de likes.
Fix Zombie Listeners	Se agrego validacion isAdded / _binding == null en FragmentCuenta y FragmentMisAnuncios para evitar crasheos.

Fin del documento